

1. Análisis del Problema

El presente trabajo plantea el desafío de seleccionar un conjunto de jugadores para disputar un partido amistoso con el objetivo estratégico de apaciguar las presiones de la prensa. Para lograrlo, el equipo técnico debe asegurarse de contentar a diferentes medios periodísticos, seleccionando al menos un jugador del agrado de cada uno de ellos, pero comprometiendo la menor cantidad de cupos posibles en el equipo.

Formalizando el escenario, podemos definir los elementos del problema de la siguiente manera:

- Un conjunto universal A conformado por los n jugadores convocados (en este caso, los 43 jugadores de la lista).
- Una serie de m subconjuntos $B_1, B_2, \dots, B_m$, donde cada $B_i \subseteq A$ representa el subconjunto de jugadores que exige ver en cancha el i -ésimo medio de prensa.

El objetivo original del problema consiste en encontrar un conjunto $C \subseteq A$ de tamaño mínimo tal que logre intersectar a todos los subconjuntos de preferencias periodísticas. Es decir, se debe garantizar la siguiente condición:

$$C \cap B_i \neq \emptyset \quad \forall i \in \{1, 2, \dots, m\} \quad (1)$$

Este escenario modela un problema clásico de la teoría de la complejidad computacional y optimización combinatoria conocido como el **Hitting-Set Problem**.

Sin embargo, para los fines de las demostraciones teóricas iniciales de este trabajo práctico (demostración de pertenencia a NP y prueba de NP-Complejidad), es necesario abstraerse de la minimización del conjunto. Por lo tanto, abordaremos y utilizaremos estrictamente la **versión de decisión** del Hitting-Set Problem.

En esta versión, se introduce una cota superior k (que en nuestro dominio representa la cantidad máxima de jugadores que el técnico está dispuesto a "gastar" para contentar a la prensa) y el problema se reduce a responder afirmativa o negativamente a la siguiente pregunta:

Dado el conjunto A de n elementos, los m subconjuntos $B_1, B_2, \dots, B_m$ de A , y un número entero k , ¿existe un subconjunto $C \subseteq A$ con $|C| \leq k$ tal que C tenga al menos un elemento de cada B_i ?

2. Demostración de Pertenencia a NP

Para demostrar que la versión de decisión del *Hitting-Set Problem* pertenece a la clase NP, debemos probar que dada una solución candidata (un certificado), es posible verificar su validez en un tiempo acotado por un polinomio.

A continuación, se presenta el algoritmo certificador escrito en Python que valida una solución propuesta:

```
1 def verificador_hitting_set(k, solucion, subsets, universo):
2     """
3     Verifica en tiempo polinomial si una solución candidata
4     es válida para el Hitting Set Problem (versión de decisión).
5     """
6
7     # 1. Verificar restricción de cardinalidad
8     if len(solucion) > k:
9         return False
10
11    # 2. Verificar que los elementos pertenezcan al universo
12    for elemento in solucion:
13        if elemento not in universo:
14            return False
15
16    # 3. Verificar condición de intersección (Hitting Set)
17    for subconjunto in subsets:
18        interseccion_encontrada = False
19
20        for elemento in solucion:
21            if elemento in subconjunto:
22                interseccion_encontrada = True
23                break # Globo pinchado, pasamos al siguiente
24
25        if not interseccion_encontrada:
26            return False
27
28    return True
```

Análisis de Complejidad

Para justificar que el validador opera en tiempo polinomial, definimos las variables que representan el tamaño de nuestra entrada:

- n : Cantidad de elementos en el universo ($|A|$).
- m : Cantidad de subconjuntos en `subsets`.
- c : Cantidad de elementos en la solución candidata ($|C|$). Por definición lógica del problema, $c \leq n$.

Evaluamos el costo temporal de cada paso del algoritmo en el peor de los casos:

1. **Verificación de cardinalidad (Línea 8):** Obtener la longitud de la solución candidata y compararla con k toma un tiempo de $\mathcal{O}(1)$ (o a lo sumo $\mathcal{O}(c)$ dependiendo de la implementación subyacente de la estructura de datos).
2. **Pertenencia al universo (Líneas 12-14):** Se itera sobre los c elementos de la solución, verificando para cada uno si pertenece al universo de tamaño n . El costo en el peor caso es $\mathcal{O}(c \cdot n)$.
3. **Condición de Hitting Set (Líneas 17-26):** Se itera sobre los m subconjuntos. Para cada uno (que puede tener hasta n elementos), se busca si contiene alguno de los c elementos de la solución. Este proceso de búsquedas anidadas requiere a lo sumo $\mathcal{O}(m \cdot c \cdot n)$ operaciones.

El tiempo total de ejecución en el peor de los casos está acotado asintóticamente por el paso más costoso: $\mathcal{O}(m \cdot c \cdot n)$. Dado que la cantidad de elementos en la solución nunca será mayor al universo total ($c \leq n$), podemos simplificar la cota superior a $\mathcal{O}(m \cdot n^2)$.

Como la función $\mathcal{O}(m \cdot n^2)$ es un polinomio respecto al tamaño de la entrada, queda formalmente demostrado que la verificación de un certificado se puede realizar en tiempo polinomial. En conclusión, el **Hitting-Set Problem se encuentra en la clase NP**. tikz

3. Demostración de NP-Compleitud

Para demostrar que la versión de decisión del problema de *Hitting-Set* es NP-Completo, debemos satisfacer dos condiciones:

1. Demostrar que pertenece a la clase NP (demostrado en la sección anterior).
2. Demostrar que un problema NP-Completo conocido se puede reducir en tiempo polinomial a *Hitting-Set*.

Para este segundo paso, elegimos reducir desde el problema de **Dominating Set** (Conjunto Dominante), el cual se sabe que es NP-Completo.

3.1. Transformación (Reducción Polinomial)

Dada una instancia genérica de *Dominating Set* compuesta por un grafo no dirigido $G = (V, E)$ y un entero k , construiremos una instancia equivalente para *Hitting-Set* de la siguiente manera:

- **Universo A :** Definimos el conjunto universal de elementos como los vértices del grafo. Es decir, $A = V$.
- **Subconjuntos B_i :** Para cada vértice $v \in V$, creamos exactamente un subconjunto B_v . Los elementos de B_v serán el propio vértice v y todos sus vecinos directos en el grafo. Matemáticamente, esto se corresponde con la vecindad cerrada de v : $B_v = N[v]$.
- **Parámetro k :** Mantenemos el mismo límite de tamaño k .

Esta construcción recorre cada vértice y sus aristas adyacentes para armar los conjuntos. Por lo tanto, su complejidad temporal está acotada por $\mathcal{O}(|V| + |E|)$, lo cual garantiza que la transformación se realiza en **tiempo polinomial**.

3.2. Ejemplo Gráfico de la Reducción

Para ilustrar la transformación, utilizaremos un grafo de ejemplo arbitrario (ver Figura 1) con 8 vértices (a, b, c, d, e, f, g, h). Este grafo presenta un nodo central y un componente aislado, lo cual es ideal para visualizar la construcción de los subconjuntos.

Al aplicar nuestra función de reducción polinomial, transformamos este grafo en una instancia de *Hitting-Set* donde el universo es $A = \{a, b, c, d, e, f, g, h\}$. Generamos un subconjunto B_v por cada vértice en base a sus vértices adyacentes ($B_v = N[v]$):

- $B_a = \{a, b, f\}$
- $B_b = \{b, a, f\}$
- $B_c = \{c, g, f\}$
- $B_d = \{d, e\}$

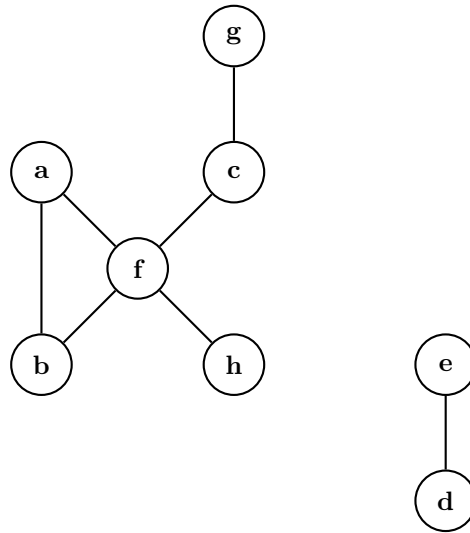


Figura 1: Grafo original de ejemplo utilizado para instanciar el problema de Dominating Set.

- $B_e = \{e, d\}$
- $B_f = \{a, b, c, f, h\}$
- $B_g = \{g, c\}$
- $B_h = \{h, f\}$

Para visualizar esta nueva instancia, podemos representar los subconjuntos como agrupaciones que encierran a sus respectivos elementos, tal como se observa en la Figura 2.

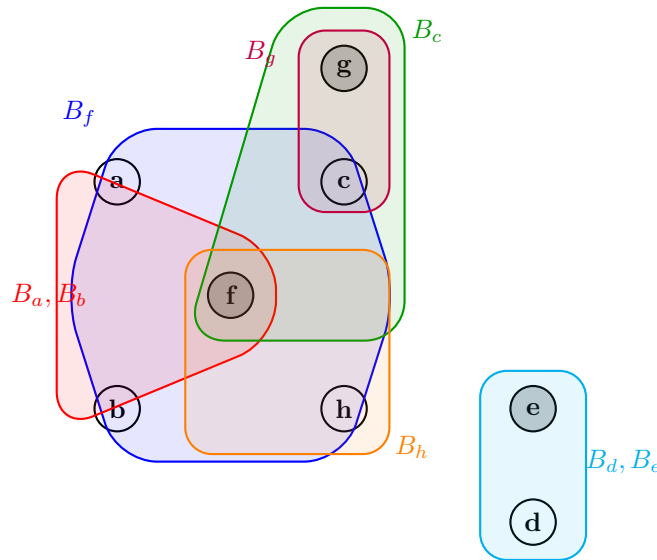


Figura 2: Representación del Hitting-Set. Los globos de colores representan los subconjuntos generados. Los nodos sombreados corresponden a la solución $C = \{f, g, e\}$, demostrando cómo intersecan a la totalidad de los subconjuntos.

Correspondencia de la solución: Si resolvemos *Dominating Set* en el grafo original (Figura 1), la solución óptima es $D = \{f, g, e\}$.

1. El nodo f domina a $\{a, b, c, f, h\}$.

2. El nodo g domina a $\{g, c\}$.
3. El nodo e domina a $\{e, d\}$.

3.3. Demostración de Correctitud (Si y solo si)

Para que la reducción sea válida, debemos demostrar que la instancia original de *Dominating Set* tiene respuesta afirmativa si y solo si la instancia construida de *Hitting-Set* también la tiene.

Si el grafo G tiene un Conjunto Dominante D de tamaño $\leq k$, entonces existe un *Hitting-Set* C de tamaño $\leq k$ para los subconjuntos construidos.

Supongamos que D es un conjunto dominante válido de G . Proponemos que nuestro *Hitting-Set* sea exactamente el mismo conjunto de vértices ($C = D$). Debemos probar que C tiene al menos un elemento en común con cada subconjunto B_v .

Tomemos un subconjunto cualquiera B_v . Por nuestra construcción, B_v contiene al vértice v y a todos sus vecinos. Como D es un conjunto dominante, sabemos por definición que el vértice v pertenece a D , o bien al menos uno de sus vecinos pertenece a D . En cualquier caso, al menos un elemento de D (nuestro conjunto C) se encuentra en B_v .

Por lo tanto, $C \cap B_v \neq \emptyset$ para todo $v \in V$, demostrando que C es un *Hitting-Set* válido de tamaño $\leq k$.

Si existe un *Hitting-Set* C de tamaño $\leq k$ para los subconjuntos construidos, entonces el grafo G original tiene un Conjunto Dominante D de tamaño $\leq k$.

Supongamos que existe un *Hitting-Set* C válido de tamaño $\leq k$. Proponemos que nuestro Conjunto Dominante sea exactamente ese conjunto ($D = C$). Debemos probar que todo vértice $v \in V$ está dominado por D .

Tomemos un vértice cualquiera $v \in V$. En nuestra instancia de *Hitting-Set*, existe un subconjunto B_v que representa la vecindad cerrada de dicho vértice. Como C es un *Hitting-Set*, sabemos que obligatoriamente interseca a B_v , lo que implica que existe un elemento $u \in C$ tal que $u \in B_v$.

Dado que los únicos elementos dentro de B_v son el propio v y sus vecinos, esto significa que el vértice u (que pertenece a D) es el propio vértice v o es adyacente a él. En ambos escenarios, el vértice v resulta dominado por un elemento de D .

Como esto se cumple para todo vértice en V , D es un Conjunto Dominante válido de tamaño $\leq k$.

Conclusión: Al haber demostrado la pertenencia a NP y haber establecido una reducción polinomial válida desde un problema NP-Completo (*Dominating Set*), se concluye formalmente que el problema de decisión de *Hitting-Set* es **NP-Completo**.

4. Resolución mediante Backtracking

Tal como concluimos en nuestro análisis teórico, al enfrentarnos a un problema NP-Completo como la selección de jugadores para contentar a la prensa (*Hitting-Set*), sabemos que no existe un algoritmo de tiempo polinomial que nos garantice la respuesta óptima. Por lo tanto, para encontrar la lista mínima exacta de jugadores convocados, debemos recurrir a técnicas de búsqueda exhaustiva.

Para este trabajo, implementamos un algoritmo de **Backtracking**. Esta técnica explora sistemáticamente todas las combinaciones posibles de jugadores (incluyéndolos o descartándolos de

la lista final). Sin embargo, para evitar que el tiempo de ejecución se vuelva inmanejable incluso para listas pequeñas, le agregamos dos optimizaciones clave:

1. **Pre-cálculo de coberturas:** Antes de empezar a explorar, armamos un diccionario que nos dice instantáneamente a qué medios de prensa contenta cada jugador.
2. **Poda por optimalidad:** A medida que exploramos, guardamos el tamaño de la mejor lista de jugadores que encontramos hasta el momento. Si en una rama de la exploración vemos que la cantidad de jugadores seleccionados ya igualó o superó a nuestro récord actual, dejamos de explorar ese camino inmediatamente, ya que es matemáticamente imposible que mejore nuestra solución.

A continuación, se presenta el código de nuestra solución en Python:

```
1 def hitting_set_backtracking(A, B):
2     """
3     A: Lista de elementos (jugadores disponibles).
4     B: Lista de sets (cada set contiene los jugadores pedidos por un medio).
5     """
6     jugadores = list(A)
7     n = len(jugadores)
8     m = len(B)
9
10    # Peor caso inicial: todos los jugadores
11    mejor_solucion = list(A)
12
13    # Para optimizar, precalculamos que conjuntos B_i cubre cada jugador
14    # conjuntos_por_jugador[j] guardara los indices de los medios que lo quieren
15    conjuntos_por_jugador = {j: set() for j in jugadores}
16    for i, subset in enumerate(B):
17        for jugador in subset:
18            if jugador in conjuntos_por_jugador:
19                conjuntos_por_jugador[jugador].add(i)
20
21    def backtrack(idx, conjunto_actual, medios_cubiertos):
22        nonlocal mejor_solucion
23
24        # CASO BASE 1: Si se cubren a todos los medios de prensa
25        if len(medios_cubiertos) == m:
26            if len(conjunto_actual) < len(mejor_solucion):
27                mejor_solucion = list(conjunto_actual)
28            return
29
30        # PODA: Si ya igualamos o superamos el tamaño de la mejor solución
31        if len(conjunto_actual) >= len(mejor_solucion):
32            return
33
34        # CASO BASE 2: Si no hay mas jugadores pero no se cubrio a todos
35        if idx == n:
36            return
37
38        jugador_actual = jugadores[idx]
39
40        # OPCION 1: INCLUIR al jugador actual
41        nuevos_medios = conjuntos_por_jugador[jugador_actual]
42        medios_cubiertos_actualizado = medios_cubiertos | nuevos_medios
43
44        conjunto_actual.append(jugador_actual)
45        backtrack(idx + 1, conjunto_actual, medios_cubiertos_actualizado)
46        conjunto_actual.pop() # Deshacer cambio (Backtrack)
47
48        # OPCION 2: NO INCLUIR al jugador actual
49        backtrack(idx + 1, conjunto_actual, medios_cubiertos)
50
51    # Llamada inicial
52    backtrack(0, [], set())
53
54    return mejor_solucion
```

Ejemplo de Ejecución

Para corroborar la correctitud de la implementación, planteamos un pequeño set de datos simulando los deseos de cuatro medios de prensa sobre un plantel de seis jugadores:

```
jugadores_convocados = {"Messi", "Roncaglia", "Mateo Messi", "De Paul", "Dibujo  
Martinez", "Scaloni_Jr"}  
  
prensa_deseos = [  
    {"Roncaglia", "Messi"},           # Medio 1  
    {"Mateo Messi"},                 # Medio 2  
    {"De Paul", "Mateo Messi"},      # Medio 3  
    {"Dibujo Martinez", "Messi"}     # Medio 4  
]
```

Al ejecutar nuestro algoritmo, el resultado obtenido es la lista ['Messi', 'Mateo Messi'], cuyo tamaño es de 2 jugadores. Esto verifica empíricamente el funcionamiento del Backtracking: con solo citar a esos dos jugadores, Scaloni se asegura de que el Medio 1 y el Medio 4 estén contentos (por Messi), y que el Medio 2 y Medio 3 también lo estén (por Mateo Messi), logrando el subconjunto mínimo óptimo.

4.1. Demostración empírica de tiempos y complejidad

El algoritmo de resolución exacta diseñado mediante *backtracking* se encuentra condicionado por dos variables principales: la cantidad de jugadores (N), que define el tamaño del universo, y la cantidad de conjuntos o medios a cubrir (M). Teóricamente, el algoritmo explora un árbol de decisión binario donde cada nodo representa la inclusión o exclusión de un jugador, lo que genera un espacio de estados en el peor de los casos de $O(2^N)$. Por otro lado, para cada configuración candidata, el algoritmo debe verificar de manera lineal la cobertura de las M restricciones. En consecuencia, la complejidad teórica global responde al orden $O(M \cdot 2^N)$.

Para validar empíricamente este comportamiento, se generaron instancias sintéticas mediante el script `datos.py` y se registraron los tiempos de ejecución ejecutando `graficos.py`. A partir de los datos recolectados, se aplicó el método de mínimos cuadrados para evaluar y contrastar las hipótesis de complejidad.

4.1.1. Análisis variando la cantidad de jugadores (N)

En primera instancia, se mantuvo fija la cantidad de conjuntos ($M = 15$) y se barrió de forma creciente la cantidad de jugadores N . Como se observa en la Figura 3, al contrastar el ajuste exponencial $O(2^N)$ contra un modelo polinomial alternativo $O(N^2)$, la curva exponencial acompaña con precisión el disparo vertical del tiempo de ejecución provocado por la explosión combinatoria.

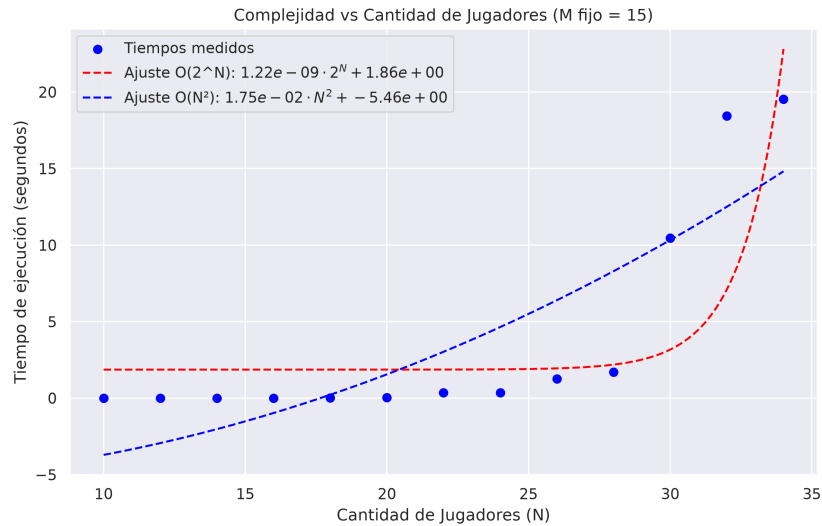


Figura 3: Tiempo de ejecución en función de la cantidad de jugadores (N).

Esta observación se convalida formalmente en la Figura 4, donde el gráfico de residuos muestra que el error absoluto del ajuste exponencial se mantiene drásticamente por debajo del error del modelo cuadrático, el cual es incapaz de modelar la aceleración asintótica del algoritmo.

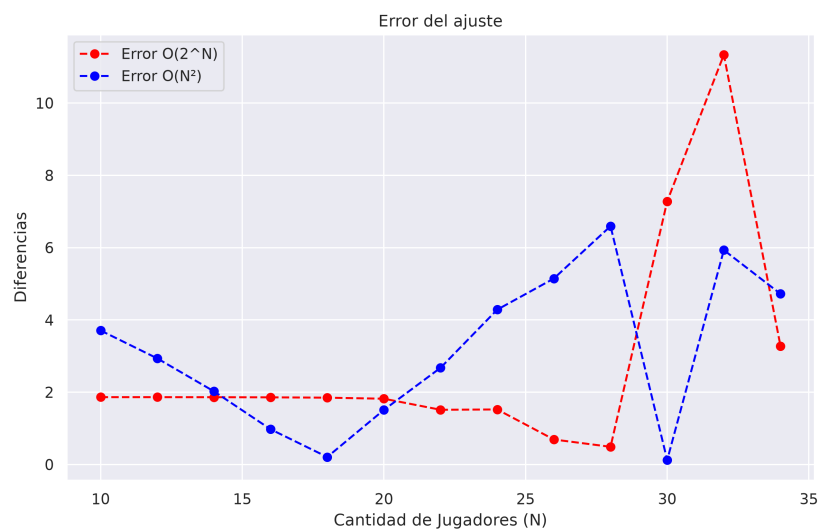


Figura 4: Error absoluto del ajuste para las hipótesis $O(2^N)$ y $O(N^2)$.

4.1.2. Análisis variando la cantidad de conjuntos (M)

Para estudiar el impacto de la cantidad de restricciones, se procedió a fijar la cantidad de jugadores en un valor constante ($N = 25$) variando el parámetro M . Dado que el tamaño del árbol de backtracking permanece fijo, el tiempo de procesamiento pasa a depender exclusivamente de las verificaciones de cobertura sobre los conjuntos.

Al contrario de la experiencia anterior, el comportamiento esperado aquí es estrictamente **lineal**, es decir, $O(M)$. En la Figura 5 se evidencia que la nube de puntos describe una trayectoria rectilínea. Al contrastar el ajuste lineal $O(M)$ contra una hipótesis cuadrática $O(M^2)$, se aprecia que la recta se superpone perfectamente sobre los datos experimentales.

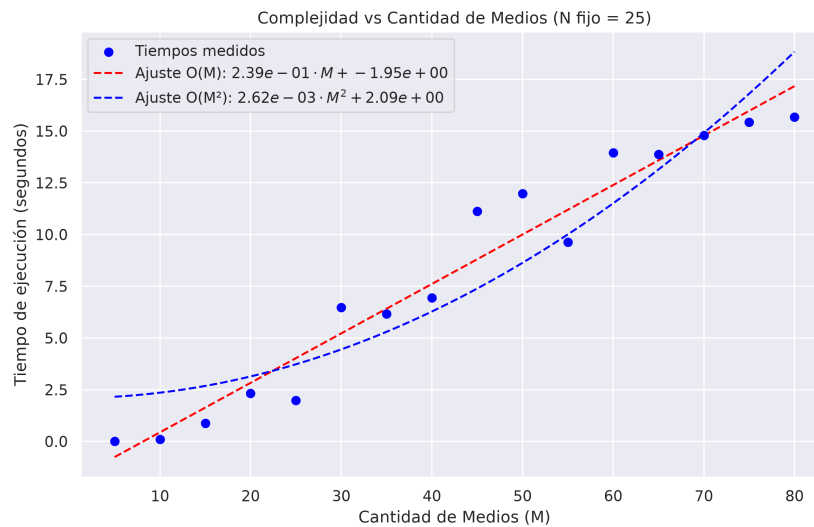


Figura 5: Tiempo de ejecución en función de la cantidad de conjuntos (M).

El análisis de residuos presentado en la Figura 6 confirma esta conclusión: mientras que el error del modelo lineal fluctúa de forma acotada y muy cercana a cero, el error del modelo cuadrático se incrementa sistemáticamente en los extremos del dominio debido a que la parábola penaliza el ajuste intentando forzar una curvatura inexistente en la realidad empírica del programa.

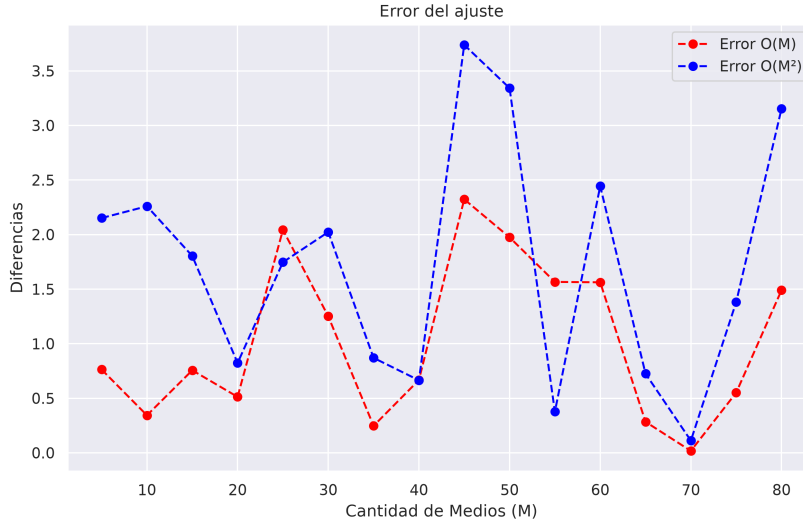


Figura 6: Error absoluto del ajuste para las hipótesis $O(M)$ y $O(M^2)$.

5. Resolución mediante programación lineal entera

Como alternativa a la búsqueda exhaustiva mediante *backtracking*, el problema de selección de jugadores para contentar a la prensa (Hitting-Set) puede modelarse matemáticamente y resolverse utilizando técnicas de programación lineal entera (ILP). Esta formulación permite aprovechar algoritmos avanzados de optimización matemática, como *simplex* y *branch-and-bound*, que resultan mucho más eficientes en la práctica para evitar tiempos de ejecución inmanejables de las instancias de mayor tamaño.

A partir de la formalización previa del escenario, donde contamos con un conjunto universal A de n jugadores y m subconjuntos $B_1, B_2, \dots, B_m$ correspondientes a las exigencias de cada medio, procedemos a definir el modelo de optimización:

Variables de decisión

Para determinar qué jugadores conformarán la lista final, definimos una variable binaria x_j para cada jugador $j \in A$. Esta variable actuará como un indicador lógico de la convocatoria del jugador:

$$x_j = \begin{cases} 1 & \text{si el jugador } j \text{ es convocado.} \\ 0 & \text{en caso contrario.} \end{cases} \quad \forall j \in A \quad (2)$$

Función objetivo

El objetivo del equipo técnico consiste en minimizar la cantidad total de cupos utilizados en el equipo. Matemáticamente, esto equivale a minimizar la suma de todas las variables de decisión (es decir, la cantidad de variables con valor 1):

$$\text{mín } Z = \sum_{j \in A} x_j \quad (3)$$

Restricciones

Para garantizar que se cumpla la condición de Hitting-Set, es obligatorio contentar a todos los medios periodísticos, seleccionando al menos un jugador de cada subconjunto. Esto se traduce en que, para cada medio i , la suma de las variables correspondientes a los jugadores requeridos en B_i debe ser mayor o igual a 1:

$$\sum_{j \in B_i} x_j \geq 1 \quad \forall i \in \{1, 2, \dots, m\} \quad (4)$$

Además, se debe declarar la naturaleza del dominio de las variables de decisión:

$$x_j \in \{0, 1\} \quad \forall j \in A \quad (5)$$

Análisis de complejidad del modelo lineal

La implementación de este modelo se encuentra en el archivo `hitting_set_lineal.py`, utilizando la biblioteca PuLP de Python. Es importante aclarar que plantear el problema con programación lineal entera (ILP) no cambia su dificultad matemática de fondo: como ya se demostró, el problema sigue siendo NP-completo.

Para resolver este modelo, las herramientas modernas (como el motor que usa PuLP por defecto) no prueban todas las combinaciones posibles a ciegas. Utilizan una mezcla del algoritmo *simplex* y una técnica de búsqueda con ramificación y poda (*branch-and-bound*). La complejidad temporal en el peor de los casos se entiende analizando estos dos pasos:

1. ***simplex***: El algoritmo *simplex* resuelve una versión relajada de forma muy rápida, aunque su peor caso teórico siga siendo exponencial.
2. ***branch-and-bound***: En el peor escenario absoluto, donde las cuentas matemáticas previas no logran ayudar a descartar malas opciones, el algoritmo terminaría explorando todas las combinaciones posibles. Para n jugadores en la lista, esto nos da una complejidad asintótica de $O(2^n)$.

En resumen, la resolución con este modelo lineal tiene un límite teórico de $O(2^n)$ en su peor caso, exactamente igual que un algoritmo de *backtracking* puro. Sin embargo, su enorme superioridad aparece en la **práctica**.

Mientras que el *backtracking* tradicional depende de reglas de poda heurísticas que nosotros programamos a mano, el modelo matemático aprovecha *simplex* para calcular límites numéricos muy estrictos. Estas herramientas matemáticas logran descartar porciones masivas del árbol de combinaciones casi al instante, permitiéndonos resolver listas con cientos de jugadores y presiones de la prensa en apenas fracciones de segundo.

Conclusión del modelo

Este conjunto de ecuaciones traduce el requerimiento de encontrar el subconjunto $C \subseteq A$ de tamaño mínimo a un formato estándar de optimización matemática. Al proveer este modelo a un *solver* matemático, este puede encontrar el tamaño k mínimo exacto sin la necesidad de explorar ineficientemente todo el árbol de combinaciones, delegando la poda a las relajaciones continuas del algoritmo *simplex*.

6. Conclusiones

A lo largo de este trabajo práctico, logramos analizar y clasificar la complejidad del problema de *Hitting-Set*. En primer lugar, construimos un algoritmo validador que nos permitió confirmar que

el problema pertenece a la clase NP. En palabras simples, esto significa que si alguien nos propone una lista de jugadores como solución, podemos verificar rápidamente (en tiempo polinomial) si esa selección cumple con las exigencias de todos los medios de prensa.

Posteriormente, demostramos que el problema es NP-Completo utilizando una técnica conocida como reducción, partiendo del problema de *Dominating Set*. La utilidad de aplicar reducciones en la computación es enorme: nos permite usar lo que ya sabemos para clasificar problemas nuevos. Como ya está demostrado que *Dominating Set* es un problema computacionalmente "difícil", al lograr transformarlo en nuestro problema de *Hitting-Set*, probamos que el nuestro hereda esa misma dificultad. Básicamente, esta técnica nos ahorra el esfuerzo de tener que analizar la dificultad del problema desde cero, dándonos una garantía matemática sobre a qué nos enfrentamos.

Llevando todo este análisis teórico al caso real (la necesidad del cuerpo técnico de seleccionar jugadores para contentar a la prensa), haber comprobado que este problema es NP-Completo tiene una consecuencia directa: es altamente improbable que exista una "fórmula mágica." o un algoritmo rápido que nos dé la lista óptima de jugadores en poco tiempo para cualquier cantidad de datos.

Saber esto justifica formalmente por qué debemos cambiar nuestra estrategia de programación en las siguientes etapas del trabajo. Para listas de convocados de tamaño moderado, no nos queda otra opción que usar métodos de búsqueda exhaustiva inteligentes, como el *Backtracking*, que analizan muchas combinaciones pero descartan rápidamente las que no sirven. Por otro lado, si en el futuro la cantidad de jugadores y de periodistas creciera muchísimo, la teoría nos avisa desde ahora que lo mejor será dejar de buscar el equipo perfecto y empezar a usar heurísticas o algoritmos de aproximación, que nos entreguen un equipo "lo suficientemente bueno."^{en} un tiempo razonable.